# CoBench Guide

## 1 CoBench intro

*from march 2026*

---

### 1.1 Purpose

CoBench is a collaborative semi-supervised anomaly detection benchmark designed for public sharing. It unifies detectors from ADBench (tabular/CV/NLP), GADBench (graph supervised), and PyGOD (graph unsupervised) under a single API so they can be evaluated both solo and collaboratively.

#### 1.1.1 Design philosophy

The benchmark does not use the original `fit()` API of integrated methods. Instead, it enforces:

- `train(epochs)` – incremental training, one or more epochs at a time. This is the backbone of the co-learning loop, which alternates between pseudo-label exchange and training. A traditional `fit()` trains the whole model at once and cannot be interrupted mid-training to receive new labels. Or maybe it could be I haven't found a way to do so.
- `get_loss(use_val=False)` – optional but recommended. Returns the latest train or validation loss. Used by early-stopping strategies and for monitoring dynamics. As we make different models collaborate it is important to synchronize their convergence.
- `predict_scores()` – returns anomaly scores in `[0, 1]`. These scores drive pseudo-label generation during the exchange phase.

Why this matters: the co-learning loop calls `train(`*`epochs`*`)` repeatedly, injecting pseudo-labels between epochs. If a model only supports `fit()`, it cannot participate in collaborative learning. Every wrapper must decompose training into epoch-level increments. However we have problems around non-differentiable methods, they cannot be updated as smoothly as gradient-based models during collaborative training. In practice, we have to fully retrain them at each collaboration step, which is costly and less clean. This is a point to work around, especially as tree and boosting methods are often SOTA.

#### 1.1.2 What the benchmark produces

For each dataset x trial, it (should) compares:

| Evaluation | Description |
|---|---|
| Solo baseline | Each detector trained alone via their native `fit()` |

| | |
|---|---|
| Collaborative | Models exchange pseudo-labels via `CoLearnerVal.cotrain()` |
| Collaborative + GRU | Same as above, with a recurrent judge on stacked embeddings |

Primary metrics: ROC-AUC and Average Precision on the held-out test set. Deltas (collab minus solo) quantify the collaboration gain.

---

## 1.2 Global Architecture

### 1.2.1 Folder structure (what actually exists)

```
CoBench/
  src/
    base.py                              # Abstract classes: Model, Data, Strategy,
    CoLearning, RecurrentModel
    colearner.py                         # Concrete co-learning loops: SimpleCoLearner,
    CoLearnerVal,
                                         #   RecurrentCoLearner,
    DelayedRecurrentCoLearner, SingleModel
    strategy.py                          # Convergence strategies: PlateauStrategy,
    AdaptivePlateauStrategy,
                                         #   RecurrentPlateauStrategy
    recurrentmodels.py                   # GRURecurrentModel (PyTorch GRU judge)
    benchmark_config.py                  # All paths, dataset registry, hyperparams, CV
    defaults
    main.py                              # CLI entrypoint for running the benchmark
    runner.py                            # Cross-validation loop (run_cv), summary
    printing, plot dispatch
    results_io.py                        # CSV export of CV results (stats + raw per-
    trial)
    utils.py                             # create_models, set_seed, validate_data,
    extract_embeddings_auto
    ADWrapperUtils.py                    # Legacy model registry (generate_AD_dictionary
    ); not used in main flow
    baselines/
      adbench/
        ModelWrapperADBench.py           # PReNetWrapper, DeepSADWrapper, DevNetWrapper,
     XGBODWrapper
                                         #   + get_model_detector_dict() registry
        data_loader.py                   # ClassicalADBenchData (Data subclass for .npz
    datasets)
      gadbench/                          # Placeholder (empty files); graph support not
    yet implemented
    results/                             # Auto-numbered CSV output files (stats + raw)
    texgad/                              # Useless for now shall contain an ablation
    study
  SubModules/                            # Unmodified clones of ADBench, GADBench, PyGOD
  docs/                                  # README_Project.md, README_API.md, NEXT_STEPS.
    md
  AGENTS.md                              # LLM agent instructions and class reference
  SUMMARY.md                             # Architecture summary
```

### 1.2.2 Module responsibilities

| Module | Role | Layer |
|---|---|---|
| `base.py` | All abstract interfaces (Model, Data, Strategy, CoLearning, RecurrentModel) + PseudoLabelProposal + default_arbiter | Core API and heart of the code |
| `colearner.py` | Concrete co-learning loops; pseudo-label exchange orchestration; evaluation | Experiment logic |
| `strategy.py` | Convergence/early-stopping policies | Experiment logic |
| `recurrentmodels.py` | GRU judge model (a RecurrentModel implementation) | Judge Model |
| `ModelWrapperADBench.py` | Wrappers adapting ADBench detectors to the Model API | Wrapper |
| `data_loader.py` | ClassicalADBenchData: .npz loading, splitting, normalization | Data handling |
| `benchmark_config.py` | Paths, dataset list, model hyperparams, CV defaults | Config |
| `main.py` | CLI argument parsing, validation, dispatch to `run_cv` | Script |
| `runner.py` | Cross-validation loop, solo+collab+GRU per trial, summary | Script |
| `results_io.py` | CSV export (stats + raw) with encoded filenames + metadata | Output |
| `utils.py` | Seed management, model factory, data validation, embedding extraction | Utils function, |

---

## 1.3 Critical Files – Hopefully never modified

### 1.3.1 `src/base.py`

We wish to build a public framework that is as clean and simple as possible. It would be best if we could add models, datasets, co-learning strategies etc easily. We define here the base rules for every class that can be customized. Every class in the system inherits from or depends on the abstract interfaces defined here. The `Model` base class defines the `y_train` property (the pseudo-label resolution cascade), the `set_pseudo_labels` / `clear_pseudo_labels` methods, and the abstract method signatures that all wrappers must implement.

#### 1.3.2 src/colearner.py

Contains the core training loops. `CoLearnerVal.cotrain()` is a basic class that relies on early stopping based on losses. `SimpleCoLearner.exchange()` implements the pseudo-label exchange protocol.

#### 1.3.3 src/benchmark_config.py

Single source of truth for all paths (PROJECT_ROOT, ADBENCH_ROOT, dataset paths), hyperparameters (MODEL_CONFIGS), and CV defaults. Every module imports from here.

#### 1.3.4 src/utils.py

`validate_splits()`, `check_no_test_leakage()`, and `check_unlabeled_hidden()` are the safety nets that catch data integrity bugs before training starts. It shall be erased when we are sure of how we manage the data, for now it's useful.

---

### 1.4 Safe Modification Zones

#### 1.4.1 Adding a new model wrapper

We need to "transfer" the models from ADBench to our benchmark, not necessarly all of them but for now simply one per main category of networks, or each model that is SOTA for at leat a dataset. `src/baselines/adbench/ModelWrapperADBench.py`

How: Create a new class inheriting from `Model` (imported from `base`).
Implement: `train(epochs)`, (`fit()` we skip fit for now, hopefully a great train + strategy replicates a fit), `predict_scores(indexes, use_train, use_val)`, `get_embeddings(indexes, use_train)`.
implement `get_loss(use_val)` for early-stopping support.
Register the class in `get_model_detector_dict()` at the top of the file.
Add default hyperparameters in `benchmark_config.py` under `MODEL_CONFIGS`.
Note : the get$_{e}mbeddingsfunctionshouldnotbebuilteverytimeasusingpytorchhooksshouldbeeasiertoworka$

def fit(self) -> None: run train(total$_{e}pochs)orwhilenot(Stoppingmechanism)train(epochsstep)$
def train(self, epochs: int = 10) -> None: y$_{t}rain = self.y_{t}rainMUSTusethisproperty, noty_{t}rain_{o}rigina$
$...trainoneepoch...self._{c}urrent_{e}poch+ = 1self._{f}itted = True$
def predict$_{s}cores(self, indexes = None, use_{t}rain = False, use_{v}al = False) - > np.ndarray :$
$Selectdatasourceifuse_{t}rain : X = self.X_{t}rainelifuse_{v}al : X = self.X_{v}alelse : X = self.X_{t}est[...]returnscores$
def get$_{e}mbeddings(self, indexes = None, use_{t}rain = True) - > np.ndarray : Returninternalrepresentationsorus$

#### 1.4.2 Adding a new convergence strategy

`src/strategy.py`
Inherit from `Strategy` (from `base.py`). Implement `should_continue(model_metrics, chapter)` and `reset()`. Register in `runner.py`'s `STRATEGY_REGISTRY` dict.

#### 1.4.3 Adding a new dataset

`src/benchmark_config.py` (add to `EVAL_DATASETS` dict)
The dataset must be a `.npz` file with either `{X, y}` or `{X_train, y_train, X_test, y_test}` keys. `ClassicalADBenchData` handles both formats. `The handling of graphs hasn't been thought yet, we should most probably follow the syntax set in GADBench`

### 1.4.4 Adding a new recurrent model

Create a new file if its "exotic" or add to `src/recurrentmodels.py` if it should appeal to the more common tests sets

Inherit from `RecurrentModel` (from `base.py`). Implement `train(aggregated_embeddings, labels, epochs)`, `predict_scores(aggregated_embeddings)`, and optionally `get_loss(aggregated_embeddi labels)`.

### 1.4.5 Adding a new co-learner variant

`src/colearner.py`

Inherit from `CoLearning`. Override `cotrain()` and/or `exchange()`. Keep the same metric dict structure so strategies and results export work.

---

## 1.5 coding rules

### 1.5.1 Interface constraints

`train(epochs)` must use `self.y_train`, never `self.y_train_original`. The `y_train` property resolves pseudo-labels on top of semi-supervised labels. Bypassing it breaks collaborative learning. We remind that each model has its own dataset (originating from train_original but evolving through pseudo labeling at eahc chapter)

`predict_scores()` returns values in `[0, 1]`. All downstream logic (threshold comparisons, AUC computation, ensemble averaging) depends on this normalization.

`predict_scores()` must support `use_train=True` and `use_val=True`. The exchange phase calls `predict_scores(use_train=True)` to score training data for pseudo-label generation. `CoLearnerVal` calls `predict_scores(use_val=True)` for validation AUC monitoring.

`get_loss(use_val=True)` should return a float or None. Strategies use this for early stopping. If your model does not track loss, return `None`. Maybe this is a bad idea and it should be imposed.

`fit()` must call `train(remaining_epochs)` to fill the total_epochs budget. This ensures the solo baseline uses the same underlying training code.

### 1.5.2 Naming conventions

- Model wrapper classes: `{ModelName}Wrapper` (e.g., `PReNetWrapper`)
- Registry keys: lowercase model name (e.g., `"prenet"`, `"deepsad"`)
- Loss history attributes: `_train_loss_history` (list of floats), `_val_loss_history` (list of floats)
- Last loss attributes: `_last_train_loss`, `_last_val_loss`

when a function is internal to a class it should start with "_"

### 1.5.3 Experiment reproducibility

- Always use `set_seed(seed)` from `utils.py` before creating data or models. It sets Python, NumPy, and PyTorch seeds. This may be subject to change later if we add a new layer of randomness (i.e. dgl seed ?).
- CV trials use `seed + trial_idx` for per-trial determinism.
- Do not use `torch.cuda.is_available()` to change model architecture; only use it for device placement.

### 1.5.4 Results and output

- Results files follow the naming convention `{id}.{n_models}.{recurrent}.{colearner}.{strategy}.{mo` It is heavy and ugly, I am open to suggestions everywhere and especially here.
- Never manually name result files; use `results_io.export_cv_results()` which auto-increments the ID.
- CSV files have a `#META:` JSON header line containing full experiment configuration.

### 1.5.5 Coding style (especially if using LLMs)

- As few functions as possible, each clear and effective.
- Maximum 1-line comments per function.

---

## 1.6 Hidden Assumptions and Invariants

### 1.6.1 Data format invariants

`labeled_indexes` and `unlabeled_indexes` must be disjoint and their union must equal `range(n_train)`, i.e. we don't "generate" unlabeled data by re-using labeled data without the label.

### 1.6.2 Pseudo-label lifecycle

At the start of each chapter, `exchange()` calls `clear_pseudo_labels()` on all models, wiping previous pseudo-labels.
Each sender model scores training data via `predict_scores(use_train=True)`. It may falsely label ground-truth labels, even if it seems counter-intuitive it could bring results.
High-confidence anomalies (score $>$ `confidence_threshold_high`) and normals (score $<$ `confidence_threshol` are proposed to receiver models via `_propose_labels()`.
`_finalize_pseudo_labels()` calls the `pseudo_label_arbiter` to resolve conflicts when multiple senders propose labels for the same sample. Default: pick the proposal with highest confidence.
Resolved labels are stored in `model._pseudo_labels` (an `np.ndarray` of shape `(n_train,)` with `-1` for no proposal).
When a model's `y_train` property is accessed, pseudo-labels overlay on top of `resolve_labels()` output. If `preserve_labeled=True`, ground-truth labeled samples are protected from overwrite.
The `_y_train_cache` is invalidated whenever `set_pseudo_labels()` or `clear_pseudo_labels()` is called.

### 1.6.3 Threshold resolution priority

For pseudo-label transfer thresholds, the priority order is:
Per-pair override: `transfer_thresholds[(sender_idx, receiver_idx)][kind]`
Per-sender default: `sender.default_confidence_high` / `sender.default_confidence_low`
Global CoLearner default: `self.confidence_threshold_high` / `self.confidence_threshold_low`

### 1.6.4 Score normalization

All `predict_scores()` implementations must return values in `[0, 1]` This is a min-max rescaling within the batch. Consequently, scores are relative within a batch, not absolute. A score of 0.9 on test data does not mean the same as 0.9 on train data. The `anomaly_threshold` (default `0.5`) is applied to training-set scores only. This is probably an issue.

### 1.6.5 Embedding aggregation for the GRU

It is probably a heavy subject to study, for now we propose these simple modes :

- `"concat"`: concatenate embeddings from all models along axis=1 -> shape `(n_samples, sum_dims)`. The GRU treats this as a single-step sequence.
- `"stack"`: pad embeddings to equal dimension, stack along axis=1 -> shape `(n_samples, n_detectors, dim)`. The GRU processes this as a multi-step sequence (one step per detector).
- `"mean"`: element-wise mean -> shape `(n_samples, max_dim)`.
- The `DelayedRecurrentCoLearner` defaults to `"stack"` and the `n_detectors` parameter of `GRURecurrentModel` must match `len(models)`.

### 1.6.6 Scaler fitting

`MinMaxScaler` is fit on training data only (including unlabeled). Test and validation data are `transform()`-ed. `check_no_test_leakage()` verifies `scaler.n_samples_seen_ == n_train`.

### 1.6.7 Strategy `metric_key`

Strategy classes look up a key in the `metrics` dict returned by the co-learning evaluation step. The dict always contains `"model_0"`, `"model_1"`, ..., `"ensemble"`, and (after GRU starts) `"gru"`.
If your strategy monitors a key that doesn't exist in the metrics dict, `PlateauStrategy.should_continue()` will raise a `ValueError`.

---

## 1.7 Common Failure Modes

We might have an issue installing the environment : pip uninstall scikit-learn pyod -y pip install –no-cache-dir scikit-learn==1.0.2 pip install –no-cache-dir pyod==1.0.9

---

## 1.8 Practical Workflow

### 1.8.1 Alpha notebook

In theory the alpha notebook is here to run the first light tests of a new model/strategy/co-learning.

### 1.8.2 When adding a new model

Write the wrapper class in `ModelWrapperADBench.py`, following `PReNetWrapper` as template.
Register it in `get_model_detector_dict()`.
Add default hyperparameters in `benchmark_config.py` under `MODEL_CONFIGS`.
Test solo first:

```
python main.py --models your_model --data_to_test annthyroid --n_trials 1
```

Test collaborative:

```
python main.py --models your_model deepsad --data_to_test annthyroid --n_trials 1
```

Verify that collab AUC is different from solo AUC (if not, pseudo-labels are probably not reaching your model – check that `train()` uses `self.y_train`).

### 1.8.3 When modifying existing logic

Run the benchmark before your change and save the output.
Make your change.
Run the benchmark again with the same seed and parameters.
Compare AUC/AP values. If they changed, understand why.

---

## 1.9 checklist

All abstract methods from `Model` are implemented: `train()`, `fit()`, `predict_scores()`, `get_embeddings()`

`train()` uses `self.y_train` (not `y_train_original`)

`predict_scores()` returns values in `[0, 1]` for all inputs (train, val, test)

`predict_scores()` handles `use_train=True` and `use_val=True`

`_current_epoch` is incremented in each training epoch

`_fitted` is set to `True` after training

Solo baseline runs without error: `python main.py -models your_model -n_trials 1`

Collaborative run produces different AUC than solo (if not, explain why)

No `np.nan` or `np.inf` in output scores

Loss tracking works (if implemented): `get_loss()` and `get_loss(use_val=True)` return valid floats

Results CSV is correctly generated in `src/results/`

No hardcoded paths; all paths use `benchmark_config.py`

Seeds are respected (running twice with the same seed gives the same result)

No unused imports or dead code introduced

Code follows the project style: minimal functions, max 1-line comments

---

## 1.10 Open Questions and Ambiguous Parts

### 1.10.1 Real issues

- The benchmark currently only supports tabular data via `ClassicalADBenchData`. Graph support (GADBench/PyGOD) is not implemented; the `baselines/gadbench/` files are empty placeholders.
- The CLI entry point is `main.py` which calls `runner.run_cv()`. The notebook `alpha.ipynb` is a parallel development/testing environment.
- DevNet requires TensorFlow; its import is guarded. If TF is not installed, `devnet` is simply absent from the registry.
- XGBOD requires PyOD; its import is also guarded.

### 1.10.2 Hidden real issues

- Exchange happens only on `unlabeled_indexes` (when `keep_truth=True`, which is the default). This means models only exchange pseudo-labels for samples where the ground truth is hidden. We want it to be possible.
- `anomaly_threshold` (default 0.5) is applied to min-max normalized training scores. Because normalization is per-batch, this threshold is effectively "above the median score" in many cases, not a calibrated probability.
- The GRU judge is evaluated on test embeddings at the end of each trial (in `runner._run_gru_trial`), but the collection uses `embeddings_use_train=False` by monkey-patching the attribute after training. This is fragile.
- `_y_train_cache` is invalidated by `set_pseudo_labels` and `clear_pseudo_labels`, but not by changes to `data.semisupervised_labels` or `data.resolve_labels`. If external code modifies semi-supervised labels after model construction, the cache may be stale.
- Each time we use a caching method seems like bad code.

### 1.10.3 Known fragilities

- `_test_ds_cache_id` in `DeepSADWrapper` caches by `id(X_batch)`. Since `id()` is the memory address, the cache breaks correctly on new arrays but might give false cache hits if a deleted array's memory is reused. In practice this is unlikely but worth noting.
- `texgad/` directory contains only `todo` placeholder files. No graph CLI exists yet.
- `configs/detectors/` and `data/` directories referenced in `README.md` do not exist. All hyperparameters are in `benchmark_config.py` and all datasets are loaded from `SubModules/ADBench/`.
- No automated tests exist. Validation is manual via CLI runs and notebook execution.

### 1.10.4 Open design questions

- How to be sure that the wrapped new models have the same performance compared to the original, knowing that sometimes papers have an obscure way of testing.
- How will graph data (node features + adjacency matrices) be integrated into the `Data` abstract class, which currently assumes flat `X_train`/`X_test` arrays?

- Should `predict_scores()` use batch-level or dataset-level min-max normalization? The current approach (per-call normalization) means scores are not comparable across different calls.
- Should the `Data` class own the pseudo-labels (as its `pseudo_labels_by_model` dict suggests in the base class) or should models own them (as the current implementation in `Model._pseudo_labels` does)? Currently the `Data` pseudo-label dict exists but is not used

---

### 1.11 Recommended reading order

| Priority | File | What to understand |
|---|---|---|
| 1 | `base.py` | Abstract API, `y_train` resolution, pseudo-label flow |
| 2 | `ModelWrapperADBench.py` | How a wrapper implements the API (use `PReNetWrapper` as reference) |
| 3 | `colearner.py` | `exchange()`, `cotrain()`, the chapter loop |
| 4 | `benchmark_config.py` | All paths, defaults, and hyperparameters |
| 5 | `runner.py` | CV loop, solo/collab/GRU trial structure |
| 6 | `strategy.py` | Early stopping logic |
| 7 | `data_loader.py` | How datasets are loaded and split |
| 8 | `utils.py` | Data validation, embedding extraction |
| 9 | `AGENTS.md` | Project coding rules and class reference |